

HTTPS Lifecycle

STEP 0 URL Parsing & HSTS Check (no network yet)

When you type `https://example.com` and hit Enter, the browser does several things **before any packet leaves your machine**: parses the URL, checks its HSTS preload list, and checks its local cache.

0.1 — Confirm what the browser parses

You can simulate the browser's parsing with `curl -v`, which shows the request line it would build.

RUN THIS:

```
$ curl -v https://example.com -o /dev/null 2>&1 | head -20
```

WHAT YOU'LL SEE:

```
* Trying 93.184.216.34:443...
* Connected to example.com (93.184.216.34) port 443
* ALPN: offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
> GET / HTTP/2
> Host: example.com
> User-Agent: curl/8.5.0
```

Notice curl resolved `example.com` to an IP, picked port **443** (because of the `https://` scheme), and used path `/` (the implicit root).

0.2 — Check the HSTS preload list

HSTS (HTTP Strict Transport Security) is a list of domains that **must** be accessed over HTTPS. If a domain is preloaded, the browser refuses HTTP even on the very first visit.

CHECK VIA CURL (LOOK FOR THE HEADER):

```
$ curl -sI https://example.com | grep -i strict
```

OR IN CHROME — CHECK THE PRELOAD LIST DIRECTLY:

```
# In Chrome address bar:
chrome://net-internals/#hsts
# Then "Query HSTS/PKP domain" → type: example.com
```

WHAT YOU'LL SEE IF IT'S PRELOADED:

```
Found:
static_sts_domain: example.com
static_upgrade_mode: FORCE_HTTPS
static_sts_include_subdomains: true
```

You can also browse the master list at `https://hstspreload.org/`.

Pentest relevance: HSTS defeats SSL stripping attacks (downgrading HTTPS → HTTP via MITM). Sites **not** in the preload list are vulnerable on the user's first-ever visit — this is what `sslstrip`, `bettercap`'s `https.proxy`, and similar tools exploit.

0.3 — Check what's in the browser's HTTP cache

If a fresh cached response exists, the browser may render it with **zero network activity**. Verify in DevTools.

```
# Open Chrome/Firefox DevTools (F12) → Network tab
# Reload the page → look at "Size" column:
```

```
# "(memory cache)" or "(disk cache)" = served from cache
# actual byte size = fetched over the network
```

For this tutorial, assume nothing is cached — proceed to DNS.

STEP 1 DNS Resolution (UDP/TCP 53, Layer 7)

The browser must convert `example.com` into an IP address. Resolution cascades through several caches before hitting the wire.

1a — Browser DNS cache

Each browser keeps its own short-lived DNS cache.

CHROME:

```
# Address bar:
chrome://net-internals/#dns
# Click "Clear host cache" to flush, then reload a site to repopulate
```

FIREFOX:

```
# Address bar:
about:networking#dns
```

WHAT YOU'LL SEE (CHROME EXAMPLE):

```
Host resolver cache:
  example.com      IPv4  93.184.216.34    TTL: 287s
  fonts.googleapis.com IPv4 142.250.74.74   TTL: 119s
  www.google.com   IPv6 2607:f8b0::200e TTL: 56s
```

1b — OS DNS cache

LINUX (systemd-resolved)

```
$ sudo resolvectl statistics
$ sudo resolvectl flush-caches # clear it
```

WINDOWS

```
> ipconfig /displaydns | findstr example.com
> ipconfig /flushdns           :: clear it
```

macOS

```
$ sudo dscacheutil -flushcache
$ sudo killall -HUP mDNSResponder
```

EXPECTED WINDOWS OUTPUT:

```
example.com
-----
Record Name . . . . . : example.com
Record Type . . . . . : 1
Time To Live . . . . . : 287
Data Length . . . . . : 4
```

```
Section . . . . .: Answer
A (Host) Record . . . : 93.184.216.34
```

1c — hosts file (static overrides)

The hosts file is checked **before** any DNS query. You can override any name here.

LINUX / macOS

\$ cat /etc/hosts

WINDOWS

> type C:\Windows\System32\drivers\etc\hosts

TEST IT WORKS — ADD A FAKE ENTRY AND PING IT:

\$ echo "127.0.0.1 mytest.local" | sudo tee -a /etc/hosts
\$ ping -c 1 mytest.local

PING mytest.local (127.0.0.1) 56(84) bytes of data.
64 bytes from localhost: icmp_seq=1 ttl=64 time=0.024 ms

Pentest relevance: Malware and rootkits often modify hosts to redirect security tools (e.g., antivirus update URLs) to 127.0.0.1 . Always inspect this file during host triage.

1d — Configured DNS server

If no cache hit, your machine sends a query to whatever DNS server was assigned (usually via DHCP).

FIND YOUR DNS SERVER:

LINUX

\$ resolvectl status | grep "DNS Server"
\$ cat /etc/resolv.conf

WINDOWS

> ipconfig /all | findstr "DNS Servers"

MAKE A MANUAL QUERY AND WATCH THE PACKET:

```
# Terminal 1 – start packet capture:
$ sudo tcpdump -i any -nn port 53

# Terminal 2 – fire a DNS query:
$ dig +noall +answer example.com A
```

DIG OUTPUT:

```
example.com.      287    IN      A       93.184.216.34
```

TCPDUMP OUTPUT (ONE ROUND TRIP, UDP/53):

```
14:30:01.234 IP 192.168.1.5.54321 > 192.168.1.1.53:
42+ A? example.com. (29)
14:30:01.241 IP 192.168.1.1.53 > 192.168.1.5.54321:
42 1/0/0 A 93.184.216.34 (45)
```

1e — Watch the full recursive resolution

If your DNS server doesn't have the answer cached, it walks the DNS hierarchy itself. The `+trace` flag makes `dig` bypass the cache and reproduce this walk so you can see it.

```
$ dig +trace example.com
```

EXPECTED OUTPUT (HEAVILY ABBREVIATED):

```
; Querying the root nameservers (.)
.      518400  IN  NS   a.root-servers.net.
.      518400  IN  NS   b.root-servers.net.
...
;; Received 525 bytes from 192.168.1.1#53 in 12 ms

; Root says: ask the .com TLD servers
com.   172800  IN  NS   a.gtld-servers.net.
com.   172800  IN  NS   b.gtld-servers.net.
...
;; Received 1188 bytes from 198.41.0.4#53 (a.root-servers.net) in 28 ms

; .com TLD says: ask example.com's authoritative server
example.com. 172800  IN  NS   a.iana-servers.net.
example.com. 172800  IN  NS   b.iana-servers.net.
;; Received 836 bytes from 192.5.6.30#53 (a.gtld-servers.net) in 45 ms

; Authoritative server answers:
example.com. 86400  IN  A    93.184.216.34
;; Received 56 bytes from 199.43.135.53#53 (a.iana-servers.net) in 67 ms
```

Three queries, three levels: root → TLD → authoritative. Each server only knows about its direct children. This delegated trust model is why DNS scales globally.

Pentest relevance: DNS is fertile ground — zone transfer attempts (`dig AXFR @ns1.target.com target.com`), subdomain enumeration (`amass` , `subfinder`), DNS tunneling (`iodine` , `dnscat2`), cache poisoning, and DNS spoofing in MITM scenarios. Knowing the resolution chain is essential for understanding where each attack inserts itself.

STEP 2 Routing Decision (Layer 3)

Now your machine has `93.184.216.34` as the destination. Is that IP on the local subnet, or does it need to go through the default gateway? The OS consults the routing table.

VIEW YOUR FULL ROUTING TABLE:

LINUX

```
$ ip route
```

WINDOWS

```
> route print -4
```

macOS

```
$ netstat -rn -f inet
```

LINUX OUTPUT:

```
default via 192.168.1.1 dev eth0 proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.5
169.254.0.0/16 dev eth0 scope link metric 1000
```

ASK THE KERNEL EXACTLY HOW IT WOULD ROUTE TO EXAMPLE.COM:

```
$ ip route get 93.184.216.34
```

```
93.184.216.34 via 192.168.1.1 dev eth0 src 192.168.1.5
cache
```

The kernel decided: "not local subnet → send via default gateway 192.168.1.1 out interface eth0 ."

CONFIRM THE ROUTE BY TRACING IT:

```
$ traceroute -n example.com # Linux / macOS
> tracert example.com      :: Windows
```

```
1  192.168.1.1      0.842 ms  <-- your gateway (hop 1)
2  10.50.0.1        4.123 ms  <-- ISP edge
3  100.64.0.1       8.451 ms
...
14 93.184.216.34    28.331 ms <-- destination reached
```

Pentest relevance: The routing table reveals network topology from *inside*. After getting a foothold, always run `ip route / route print` — you'll see additional internal subnets, VPN routes, and pivoting opportunities the perimeter scan never showed.

STEP 3 ARP for the Gateway (Layer 2)

The packet is going to 93.184.216.34 , but at the Ethernet layer the frame must be addressed to the gateway's MAC — that's the next hop. If we don't have it cached, we ARP for it.

VIEW YOUR CURRENT ARP CACHE:

```
$ ip neigh # Linux (modern)
$ arp -a  # Universal — Linux, Windows, macOS
```

OUTPUT:

```
192.168.1.1 dev eth0 lladdr 11:11:11:11:11:11 REACHABLE
192.168.1.20 dev eth0 lladdr aa:bb:cc:dd:ee:ff STALE
192.168.1.50 dev eth0 lladdr 22:33:44:55:66:77 REACHABLE
```

FORCE A FRESH ARP REQUEST AND WATCH IT ON THE WIRE:

```
# Terminal 1 — capture ARP only:
$ sudo tcpdump -i eth0 -nn arp

# Terminal 2 — flush the cache and ping the gateway:
$ sudo ip neigh flush all
$ ping -c 1 192.168.1.1
```

TCPDUMP OUTPUT (THE ARP EXCHANGE):

```
14:31:05.001 ARP, Request who-has 192.168.1.1 tell 192.168.1.5, length 28
14:31:05.002 ARP, Reply 192.168.1.1 is-at 11:11:11:11:11:11, length 28
```

That's it — two packets. Now your OS can build the full Ethernet frame:

```
| Ethernet header
```

Source MAC:	AA:AA:AA:AA:AA:AA	(your NIC)
Destination MAC:	11:11:11:11:11:11	(gateway – from ARP)
IP header		
Source IP:	192.168.1.5	
Destination IP:	93.184.216.34	(example.com – from DNS)
TCP header + payload ...		

Key insight: the destination IP stays as 93.184.216.34 for the entire journey, but the destination MAC **changes at every hop**. Each router rewrites it for the next leg. ARP is what made that first leg possible.

Pentest relevance: this is where ARP spoofing inserts itself. The attacker sends forged ARP replies to make the victim believe the gateway's MAC is the attacker's MAC. Every "outbound" packet now goes to the attacker first → classic MITM. Tools: arpspoof , bettercap arp.spoof , ettercap .

STEP 4 TCP 3-Way Handshake (Layer 4)

Before any HTTPS data flows, TCP must establish a reliable connection. Three packets: SYN → SYN-ACK → ACK.

CAPTURE ONLY TCP/443 TRAFFIC TO/FROM EXAMPLE.COM:

```
# Terminal 1 – capture:
$ sudo tcpdump -i any -nn -S 'host example.com and tcp port 443' -c 6

# Terminal 2 – trigger a connection:
$ curl -s https://example.com -o /dev/null
```

TCPDUMP OUTPUT — THE HANDSHAKE:

```
14:32:10.001 IP 192.168.1.5.54422 > 93.184.216.34.443:
           Flags [S],      seq 100, win 64240      <-- SYN
14:32:10.025 IP 93.184.216.34.443 > 192.168.1.5.54422:
           Flags [S.],      seq 500, ack 101       <-- SYN-ACK
14:32:10.026 IP 192.168.1.5.54422 > 93.184.216.34.443:
           Flags [.],      ack 501                <-- ACK
; connection established – TLS handshake begins next
```

Flag notation in tcpdump: S = SYN, . = ACK, S. = SYN+ACK, F = FIN, R = RST, P = PSH.

VERIFY THE CONNECTION IS ESTABLISHED:

```
$ ss -tn 'dport = :443'      # Linux
> netstat -an | findstr ":443"  :: Windows
```

State	Recv-Q	Send-Q	Local Address	Peer Address
ESTAB	0	0	192.168.1.5:54422	93.184.216.34:443

What just happened: A bidirectional, reliable TCP channel now exists between your machine and example.com's server. Both ends track sequence numbers, window sizes, and connection state in their respective kernels. From here, anything sent is guaranteed delivery (with retransmission) and in-order.

Pentest relevance: this is exactly what an nmap -sS SYN scan interrupts — Nmap sends the SYN, observes the SYN-ACK (or RST), then sends RST instead of completing with ACK. That's why it's called "half-open." Also: TCP/IP stack fingerprinting (window sizes, options, TTL, MSS) is how nmap -O identifies the OS.

The TCP channel exists, but it's plaintext. Before any HTTP data crosses, TLS sets up encryption. We'll walk through the four sub-steps: Client Hello, Server Hello, certificate validation, and key derivation.

The best tool to observe the whole handshake is `openssl s_client` — it lets you do a TLS connection manually and see every step.

5a — Client Hello

Your browser proposes: supported TLS versions, cipher suites, the target hostname (SNI), and a "key share" for ephemeral key exchange.

TRIGGER AND OBSERVE THE CLIENT HELLO:

```
$ openssl s_client -connect example.com:443 -servername example.com -msg < /dev/null 2>&1 | head -30
```

OUTPUT (CLIENT HELLO):

```
>>> TLS 1.3, Handshake [length 0200], ClientHello
 16 03 01 02 00 01 00 01 fc 03 03 ...
; ClientRandom: 32 random bytes
; Cipher Suites: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, ...
; Extension: server_name (SNI) = "example.com"
; Extension: supported_versions = TLS 1.3, TLS 1.2
; Extension: key_share = X25519 public key
```

Pentest relevance: SNI is sent **in cleartext** even in TLS 1.3 (unless ECH/Encrypted Client Hello is used — still rare in 2026). This means a passive observer on the network can see which hostname you're visiting, even if they can't see the content. Tools like Suricata and corporate firewalls use SNI for traffic filtering.

5b — Server Hello

The server picks one of the client's offered cipher suites, sends its own random, its key share, and (in TLS 1.2) starts the certificate exchange.

OUTPUT CONTINUES (SERVER HELLO):

```
<<< TLS 1.3, Handshake [length 007a], ServerHello
; Cipher Suite chosen: TLS_AES_256_GCM_SHA384
; ServerRandom: 32 random bytes
; key_share = X25519 public key
; supported_versions = TLS 1.3
```

QUICKLY CHECK WHICH CIPHER AND TLS VERSION WERE NEGOTIATED:

```
$ echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | grep -E "Protocol|Cipher"
```

```
Protocol : TLSv1.3
Cipher   : TLS_AES_256_GCM_SHA384
```

WANT TO SEE WHAT THE SERVER ACTUALLY SUPPORTS? USE NMAP OR TESTSSL:

```
$ nmap --script ssl-enum-ciphers -p 443 example.com
$ testssl.sh example.com
```

Pentest relevance: TLS misconfigurations all surface here — outdated versions (TLS 1.0/1.1, SSLv3), weak ciphers (RC4, 3DES, NULL, EXPORT-grade), Heartbleed-era OpenSSL, POODLE, ROBOT, downgrade attacks. `testssl.sh` is the standard tool for TLS configuration audits.

5c — Certificate validation

The server sends its X.509 certificate chain. Your browser/curl/openssl verifies it before continuing.

DOWNLOAD AND INSPECT THE SERVER'S CERTIFICATE:

```
$ openssl s_client -connect example.com:443 -servername example.com -showcerts < /dev/null 2>/dev/null | openssl x509 -noout -text | head -25
```

OUTPUT (KEY FIELDS):

```
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number: 0x0c...
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=US, O=DigiCert Inc, CN=DigiCert Global G2 TLS RSA SHA256 2020 CA1
    Validity
      Not Before: Jan 30 00:00:00 2026 GMT
      Not After : Mar  1 23:59:59 2027 GMT
    Subject: C=US, ST=California, L=Los Angeles, O=Internet Corporation for Assigned Names and Numbers, CN=www.example.org
    X509v3 Subject Alternative Name:
      DNS:www.example.org, DNS:example.com, DNS:example.net, ...
```

WATCH OPENSsl VERIFY THE CHAIN IN REAL TIME:

```
$ openssl s_client -connect example.com:443 -servername example.com -verify_return_error < /dev/null 2>&1 | grep -E "verify|Verification"
```

```
depth=2 C=US, O=DigiCert Inc, CN=DigiCert Global Root G2
verify return:1
depth=1 C=US, O=DigiCert Inc, CN=DigiCert Global G2 TLS RSA SHA256 2020 CA1
verify return:1
depth=0 ... CN=www.example.org
verify return:1
Verification: OK
```

Validation checks performed by your client:

- ✓ Signed by a trusted Certificate Authority in the trust store
- ✓ Subject CN or SAN matches **example.com**
- ✓ Within validity period (Not Before / Not After dates)
- ✓ Not revoked (checked via OCSP, CRL, or OCSP stapling)
- ✓ Satisfies Certificate Transparency (CT log entries present)
- ✓ Key usage and extended key usage appropriate

VIEW THE TRUST STORE ON YOUR SYSTEM:

```
$ ls /etc/ssl/certs/ # Linux trust store
$ security find-certificate -a /Library/Keychains/System.keychain # macOS
> certutil -store -enterprise Root :: Windows
```

Pentest relevance: any failure here triggers the browser's "Your connection is not private" page. Attackers try to bypass this through: **(1)** tricking users into clicking through, **(2)** compromising a CA (rare but historically devastating — DigiNotar 2011), **(3)** installing a malicious root CA on the victim's machine (e.g., Superfish, corporate MITM proxies, malware), **(4)** using mis-issued certificates, **(5)** attacking apps with broken pinning. Always check `/etc/ssl/certs/` for unauthorized root CAs during host triage.